

Key-Value Store v2

Persistenza locale, crash recovery e coerenza interna

Architetture dei Sistemi Distribuiti

Lezione 11

Dopo concorrenza e sezioni critiche, la domanda cambia:

quando una scrittura puo' essere considerata davvero confermata?

Con la sola RAM la risposta era povera. Con la persistenza locale entra in gioco un nuovo attore:

- il disco;
- la possibilita' di crash;
- la recovery dopo riavvio.

Due Stati Da Distinguere

Da questa tappa conviene distinguere:

- **stato attivo**: quello residente in RAM e usato per servire le richieste;
- **stato ricostruibile**: quello che il sistema puo' recuperare dopo crash.

Il problema nasce quando i due non evolvono nello stesso ordine logico.

Se il server:

- ➊ aggiorna la RAM;
- ➋ risponde OK;
- ➌ persiste piu' tardi;

allora esiste una finestra in cui:

- il client ha gia' ricevuto una promessa;
- il sistema non e' ancora in grado di difenderla dopo il riavvio.

Variante Unsafe

La prima variante del laboratorio fa:

- 1 update del dizionario in RAM;
- 2 OK al client;
- 3 snapshot periodico in background.

Vantaggio:

- scritture veloci.

Svantaggio:

- scritture ackate ma non ancora recoverable.

```
with self._lock:  
    self._data[key] = value  
    self._dirty = True  
return "OK", False
```

Lo snapshot e' rinviato:

```
while not self._stop_event.wait(2.0):  
    self.flush_snapshot()
```

Il punto non e' che il codice sia brutto. Il punto e' che il contratto risultante e' debole.

L'automa astratto della write e':

- AcquireLock -> ApplyInMemory -> Reply -> SnapshotLater

Osservazione importante:

- il punto in cui il client vede OK precede il punto in cui il sistema diventa recoverable.

Perche' E' Una Violazione Di Safety

Il problema non e' la perdita di performance. Il problema e' la rottura di una promessa implicita:

- il client puo' ragionevolmente interpretare OK come scrittura "fatta";
- dopo crash, quella scrittura puo' sparire;
- quindi il sistema non sa piu' difendere la propria risposta.

La seconda variante usa un write-ahead log:

- 1 serializzare l'intento di update;
- 2 appendere il record al log;
- 3 forzarlo su disco con `fsync`;
- 4 applicare l'update alla RAM;
- 5 rispondere OK.

Ora il punto di commit logico si sposta.

```
with self._lock:
    self._append_record({
        "op": "SET",
        "key": key,
        "value": value,
    })
    self._data[key] = value
return "OK", False
```

La parte decisiva e' che `_append_record` non si limita a scrivere: rende il record difendibile via `fsync`.

Nella variante safe possiamo difendere un invariante preciso:

Invariante

Ogni scrittura che ha ricevuto OK deve essere ricostruibile dopo replay del log.

Questo non implica che RAM e disco coincidano sempre. Implica che il disco contenga abbastanza informazione da sostenere le promesse fatte.

L'automa diventa:

- AcquireLock -> AppendLog -> Fsync -> ApplyInMemory -> Reply

Qui il punto chiave e':

- prima di Fsync, nessun OK e' ancora difendibile;
- dopo Fsync, il crash puo' distruggere il processo ma non il significato della scrittura;
- il replay del log colma il gap rispetto alla RAM.

Al riavvio la variante safe rilegge il log:

```
for raw_line in log_file:  
    record = json.loads(raw_line)  
    self._apply_record(record)
```

La recovery non e' un dettaglio di manutenzione. E' parte integrante del contratto: senza replay corretto, il significato di OK resterebbe ambiguo.

Safety:

- nessuna scrittura ackata deve sparire dalla storia recoverable;
- il replay non deve produrre stati impossibili.

Liveness:

- `fsync` aumenta la latenza;
- trattenere il lock durante persistenza riduce parallelismo;
- un disco lento puo' rallentare l'intero servizio.

Che Cosa Cambia Senza Cambiare API

Il client continua a mandare:

```
SET course distributed-systems
```

Ma il contenuto semantico di OK cambia:

- prima: update locale in RAM;
- ora: update che il sistema puo' ricostruire dopo crash locale.

Variante unsafe:

- 1 SET course distributed-systems
- 2 CRASH
- 3 restart
- 4 GET course

Variante safe:

- 1 stessa sequenza;
- 2 esito atteso diverso.

Un sistema puo' sembrare corretto finche' resta vivo e comunque essere semanticamente debole. La persistenza locale obbliga a distinguere:

- stato visibile adesso;
- stato ricostruibile domani;
- punto preciso in cui una risposta positiva diventa legittima.